Lecture 4: Numbers and Operators

Dr. Aafaq · B.Tech. CSE, 5th Semester · Week 2, Lecture 1 · Compact notes

1 Four Kinds of Numbers

Type	Examples	Facts
`int`	`42, -7, 1_000_000, 0b1010, 0o17, 0xFF`	Exact, **unlimited size** (`2**100` never overflows). `bin()`, `oct()`, `hex()` convert to base strings.
`float`	`9.81, 2.0, .5, 3e8, 1.602e-19`	64-bit IEEE 754: about 15 to 17 significant digits, range $\pm 1.8 \times 10^{308}$, beyond that `inf`. **Approximate.**
`complex`	`3+4j, 1j`	`.real`, `.imag`, `abs(z)`, `.conjugate()`. Python writes *j* like engineers.
`bool`	`True, False`	Subtype of `int`: `True == 1`, so `True + True` is `2`.

Example (AC circuit): `Z = 3 + 4j` ($R = 3\,\Omega$, $X_L = 4\,\Omega$); `abs(Z)` is `5.0`; on 230 V, `abs(230 / Z)` is `46.0` A. Check any value with `type(x)`.

2 Arithmetic Operators

Op	Name	Example	Result
`+ - *`	add, sub, mul	`7 * 2`	`14`
`/`	true division	`7 / 2`	`3.5`
`//`	floor division	`7 // 2`	`3`
`%`	modulo	`7 % 2`	`1`
`**`	power	`2 ** 0.5`	`1.414...`

Result type: int op int gives int, *except* `/`, which always gives a float. Any float makes the result float; any complex makes it complex. `^` is *not* power (it is XOR).

Identity: `a == (a // b) * b + a % b`.
17 samosas, 5 friends: `17 // 5` is 3 each, `17 % 5` is 2 left; `divmod(17, 5)` gives `(3, 2)`.
Negatives: `//` rounds *down* (floor), not toward zero: `-7 // 2` is `-4`, `-7 % 2` is `1` (sign of the divisor). C gives `-3` and `-1`.

```
t = 3725                          # seconds
h, m, s = t // 3600, t % 3600 // 60, t % 60
arrive = (22 + 5) % 24            # 3 o'clock
last, rest = 2026 % 10, 2026 // 10  # 6, 202
```

Key idea. `%` wraps around (clocks, weekdays, angles, even/odd with `x % 2 == 0`); `//` groups (pages, boxes). Ceiling division: `(n + k - 1) // k`.

3 Precedence (High to Low)

`( )` > `**` > unary `+x -x` > `* / // %` > `+ -` > `== != < > <= >=` > `not` > `and` > `or`. Same level runs left to right, **except `**`**, which runs right to left.

```
-3 ** 2 → -9      2 ** 3 ** 2 → 512     100 / 10 / 5 → 2.0     2 + 3 * 4 ** 2 → 50
(-3) ** 2 → 9     (2 ** 3) ** 2 → 64    17 % 5 * 2 → 4         not 2 + 2 == 5 → True
```

Classic bug: average written `a + b / 2`; correct is `(a + b) / 2`. When in doubt, add brackets.

4 Comparison, Logical and Assignment Operators

- `== != < > <= >=` return a `bool`. `=` stores, `==` asks. `5 == 5.0` is `True`.
- **Chaining:** `40 <= marks < 75` means `40 <= marks and marks < 75`.
- `and`, `or`, `not` are words (C uses `&& || !`).
- **Short-circuit:** `False and ...` and `True or ...` skip the right side, so `x != 0 and 10 / x > 2` never divides by zero.

```
passed = theory >= 30 and practical >= 20
alarm  = smoke or temp > 60
balance = 1000
balance += 500      # same as balance = balance + 500
balance *= 1.05     # also -= /= //= %= **=
x, y = y, x         # swap in one line
```

No ++: `count++` is a SyntaxError; write `count += 1`.

5 Type Conversion

Implicit (widening): bool → int → float → complex, e.g. `3 + 2.5` is `5.5`. A string is **never** converted silently: `'5' + 3` is a `TypeError`.
Explicit: `int(3.99)` is `3` and `int(-3.99)` is `-3` (chops toward zero); `float(7)` is `7.0`; `int('42')`; `float('9.81')`; `int('7.5')` is a `ValueError` (use `int(float('7.5'))`).

`int(x)`	chop toward zero
`round(x, n)`	nearest; **ties to even**: `round(2.5)` is 2, `round(3.5)` is 4
`math.floor(x)`	down: `floor(-2.5)` is −3
`math.ceil(x)`	up: `ceil(2.1)` is 3

Key idea. `input()` always returns a `str`. Write `l = float(input('Length: '))` before doing arithmetic; otherwise `l + w` silently gives `'43'`.

Dr. Aafaq

6 Floating Point Surprises

`0.1 + 0.2` gives `0.30000000000000004`: floats are stored in **binary**, where 0.1 is a never-ending fraction (like $1/3 = 0.333\ldots$ in decimal), so the nearest 64-bit value is stored. Every language does this. In 1991 a Patriot missile battery's clock, counting tenths of a second, drifted 0.34 s after 100 hours and missed a Scud; 28 soldiers died. **Never compare floats with ==**: use `math.isclose(a, b)` or `abs(a - b) < 1e-9`. For exact money use integer paise or `Decimal('0.1')`; for exact fractions `Fraction(1, 3)`. Rule of thumb: `int` when you **count**, `float` when you **measure**.

7 Common Mistakes

You wrote	What happens	Fix
`2 ^ 3` for power	`1` (bitwise XOR)	`2 ** 3`
`a + b / 2` for an average	divides only `b`	`(a + b) / 2`
`x = input(); x * 2`	repeats the string (`'55'`)	`x = int(input())`
`if total / n == 0.3`	almost never true	`math.isclose(total / n, 0.3)`
`count++`	SyntaxError	`count += 1`
`j` for the imaginary unit	NameError	`1j`
`int('7.5')`	ValueError	`int(float('7.5'))`
`true`, `false`	NameError	`True`, `False`

8 Fun to Try at Home

The 24 game: using each number once and `+ - * / ( )`, make Python print 24 from (1, 2, 3, 4), (2, 2, 5, 7), (4, 4, 10, 10), (1, 3, 4, 6) and (3, 3, 8, 8). The last one needs fractions, and Python prints `23.99999999999999`. **Number detective:** for any 4-digit `n`, `n % 10` is the last digit, `n // 1000` the first, `n // 10 % 100` the middle two. **Big numbers:** how many digits does `2 ** 1000` have? (`len(str(2 ** 1000))` is 302.)

9 Exercises

1. Give the value and type: `9 / 3`, `9 // 2.0`, `2 ** -2`, `True * 3.0`, `1e2`.
2. Evaluate by hand: `-2 ** 2 + 10 % 4`, `-9 // 4`, `-9 % 4`, `3 < 5 < 4`.
3. With `n = 4739`, write expressions for the first digit, the middle two digits and the digit sum.
4. Write one expression that is `True` for a leap year.
5. Why is `8 / (3 - 8 / 3)` not exactly 24 in Python? How should you test it?
6. (Lab) Read an amount in rupees and print the number of 500, 200 and 100 notes needed, using `//` and `%`.

Answers. 1: `3.0` float, `4.0` float, `0.25` float, `3.0` float, `100.0` float. 2: -2, -3, 3, `False`. 3: `n // 1000`, `n // 10 % 100`, `n//1000 + n//100%10 + n//10%10 + n%10` (23). 4: `y % 4 == 0 and y % 100 != 0 or y % 400 == 0`. 5: `8/3` is stored approximately, giving `23.99999999999999`; use `math.isclose(x, 24)`. 6: `n500 = amt // 500; amt %= 500; n200 = amt // 200; amt %= 200; n100 = amt // 100`.

Next: Lecture 5, Strings. Demo: `lecture04_demo.py`.

Draft